

i need the final layer 9 first, before we move to the final layer 10. again as scrum master style version, including user story, concretizations, 'what each step should do, acceptance criteria and proposed Tests and summar, remember to do adapt it my actual research and ai engineering needs.

Nachgedacht für 4s

Bundle 9 — Atomic Zettelkasten / PKM writing machine

Scrum-master view

Bundle objective: prepare a modified atomic Zettelkasten workspace for your research, AI engineering, experiment planning, and future paper writing.

This bundle is not a generic Obsidian setup. It is a structured PKM system for the platform you are building:

Research sources

- > source notes
- > atomic concept notes
- > molecule insight notes
- > research-question/topic maps
- > alloy publishable notes
- > later LaTeX section drafts

Your uploaded Zettelkasten diagram defines the important flow: capture source-linked notes, distill them into singular concepts or "atoms," combine those into insight/permanent notes or "molecules," then combine those into publishable "alloys" aimed at a publication goal.

This bundle should support your real work:

GRN / NCA / ART research
 Runpod and remote model infrastructure
 Agentfield / Paperclip platform architecture
 OpenClaw / PKM reasoning
 MRes paper development
 future product/IP notes

It should not build the paper. That is Bundle 10.

User story

As the Research Scientist / AI Engineer / Publisher,
I want a structured atomic Zettelkasten workspace,
so that sources, technical ideas, experiment notes, architecture decisions,
research questions, and publishable insights can move from rough capture
into reusable paper-ready knowledge without becoming scattered notes.

Concretizations / managed steps

```
prepare_obsidian_vault_access  
prepare_obsidian_vault_mount  
check_obsidian_vault_access  
  
prepare_atomic_zettelkasten_structure  
prepare_source_note_templates  
prepare_atom_note_templates  
prepare_molecule_note_templates  
prepare_topic_question_templates  
prepare_alloy_publish_note_templates  
prepare_latex_section_note_templates  
  
prepare_figure_export_paths  
prepare_latex_template_binding
```

I would **not** keep the older generic steps:

```
prepare_paper_note_structure  
prepare_literature_note_structure
```

They are too generic. The Zettelkasten-specific steps replace them.

What each step should do

`prepare_obsidian_vault_access`

Defines where the PKM vault lives and records that path in config policy.

Possible paths:

```
/workspace/pkm/obsidian  
/workspace/pkm/zettelkasten  
/mnt/research/obsidian  
/home/publisher/Obsidian
```

Recommended default:

```
/workspace/pkm/zettelkasten
```

because this makes it portable across local and Runpod-style workspaces.

What this does for the work: gives your notes one stable home that can later be used by OpenClaw, local model reasoning, Publisher, and Paperclip-visible review workflows.

`prepare_obsidian_vault_mount`

Prepares mount points and mount metadata for a vault that may live on local disk, SMB, external drive, or synced storage.

It should create mount directories only. It should not automatically mount or sync unless you explicitly run a separate mount/sync command.

What this does for the work: lets PKM access become repeatable without risking accidental overwrites or pulling private notes into the wrong place.

`check_obsidian_vault_access`

Non-mutating check.

Should report:

```
vault path
exists / missing
readable / not readable
writable / not writable
expected folder structure present / missing
template folder present / missing
figure bridge present / missing
LaTeX binding present / missing
```

It must not print note contents.

What this does for the work: confirms the vault is usable before you try to write notes, index context, or export paper sections.

`prepare_atomic_zettelkasten_structure`

Creates the core folder structure.

Recommended structure:

```
<vault>/
  00_inbox/
    fleeting/
    quick_capture/

  10_reference_manager_kasten/
    sources/
    source_notes/
    citations/
    quotes/
    zotero_exports/
    youtube_notes/
    papers/
    chats/
    code_outputs/

  20_knowledge_kasten/
    atoms/
    molecules/
    topics/
    research_questions/
    concepts/

  30_publish_kasten/
    alloys/
    sequences/
    publication_goals/
    paper_drafts/

  40_experiments/
    grn/
    nca_art/
    parameter_search/
    runpod/
    agentfield/
    paperclip_adapter/
    openclaw/

  50_figures/
    grn/
    nca_art/
    platform/
    runpod/
    paperclip/
    agentfield/

  60_templates/
    source_note.md
    atom_note.md
    molecule_note.md
```

```

topic_note.md
research_question.md
alloy_note.md
publication_goal.md
latex_section_note.md
figure_note.md
experiment_note.md
architecture_decision.md

```

```

70_bridge/
  latex/
  paperclip/
  agentfield/
  openclaw/

```

```

90_archive/

```

What this does for the work: creates a PKM structure that matches your actual platform: sources, research insights, experiments, figures, publishing, and platform architecture.

prepare_source_note_templates

Creates templates for rough but source-linked notes.

Templates should cover:

```

paper source note
PDF source note
YouTube/source note
chat/source note
code-output note
quote note
citation note

```

Each source note should include:

```

---
type: source
source_kind:
source_title:
source_ref:
citation_key:
related_topics:
related_research_questions:
status: inbox
---

```

```
# Source

# Raw notes

# Quotes

# Own-word summary

# Possible atoms to extract
```

What this does for the work: every idea from literature, chats, PDFs, Runpod notes, Agentfield notes, or code analysis keeps its origin. That matters later when you write the MRes paper or defend why a design choice exists.

prepare_atom_note_templates

Creates templates for atomic concept notes.

An atom is one singular concept in your own words.

Example atom themes for your project:

```
5-node GRN
Turing instability
PDE/ODE simulator
NCA surrogate
ART prototype
symbolic DSL
Runpod portable runtime
Agentfield experiment controller
Paperclip-Agentfield adapter
OpenClaw PKM query
```

Template:

```
---
type: atom
concept:
source_refs:
citation_keys:
topics:
research_questions:
status: active
---

# Concept
```

```
# In my own words

# Why this matters

# Links
- Source:
- Topic:
- Research question:
- Candidate molecule:
```

What this does for the work: turns raw sources into reusable, paper-safe concepts.

prepare_molecule_note_templates

Creates templates for permanent insight notes.

A molecule combines atoms with your own reasoning.

Example molecule themes:

```
Why config must stay below Agentfield
Why Runpod should use Docker base + config policy
Why NCA is not the same as the PDE/ODE simulator
Why ART prototypes need symbolic DSL mapping
Why Paperclip needs an adapter instead of replacing Agentfield
```

Template:

```
---
type: molecule
atoms:
source_refs:
topics:
research_questions:
experiment_links:
paper_targets:
status: developing
---

# Insight

# Atoms used

# My reasoning

# What this changes in the project

# Experiment relevance

# Publication relevance
```

What this does for the work: gives you a place to develop actual arguments, not just collect facts.

prepare_topic_question_templates

Creates templates for topic maps and research-question maps.

Topic examples:

```
[[GRN discovery]]
[[NCA-ART pipeline]]
[[Runpod runtime]]
[[Agentfield control plane]]
[[Paperclip adapter]]
[[OpenClaw PKM]]
[[MRes paper]]
```

Research-question examples:

```
Can a 5-node GRN generate stable Turing-like patterns?
Can ART prototypes summarize useful local states from PDE/ODE simulations?
Can NCA surrogates accelerate candidate screening?
Can Agentfield orchestrate a GRNExperiment lifecycle?
Can Paperclip expose the experiment lifecycle to a human operator?
```

Template:

```
---
type: research_question
question:
topic:
status:
related_atoms:
related_molecules:
related_experiments:
publication_goal:
---

# Question

# Why it matters

# Current answer

# Evidence

# Open experiments
```


# Notes for paper

What this does for the work: keeps the project focused on answerable research questions rather than endless infrastructure building.

prepare_alloy_publish_note_templates

Creates publishable-note templates.

An alloy is a sequenced publishable unit: atoms + molecules + source trail + argument direction.

Example alloy themes:

```
Config-to-Agentfield architecture
Runpod portable runtime base
NCA-ART-GRN discovery pipeline
PDE/ODE to symbolic DSL workflow
Paperclip-Agentfield adapter architecture
OpenClaw PKM writing workflow
```

Template:

```
---
type: alloy
publication_goal:
target_section:
atoms:
molecules:
citation_keys:
figures:
tables:
status: draft
---

# Claim

# Argument sequence

# Supporting atoms

# Supporting molecules

# Sources and citations

# Figures or tables needed

# Export target
```

What this does for the work: prepares material that can later become Introduction, Methods, Results, Discussion, or Architecture sections.

prepare_latex_section_note_templates

Creates note templates aligned with your actual LaTeX report direction.

Your LaTeX project uses modular section files, where the main `.tex` file inputs section files from `files/<article-type>/`, as shown in `IBA-protocol.tex`: abstract, introduction, materials, results, definitions, and risk assessment are loaded as separate files before the document body uses them.

Templates should map PKM notes to likely future LaTeX sections:

```
abstract note
introduction note
background note
methods note
simulator note
nca-art-pipeline note
parameter-search note
results note
discussion note
conclusion note
data-availability note
figure-caption note
table-caption note
```

Template:

```
---
type: latex_section_note
target_paper:
target_latex_file:
source_alloys:
citation_keys:
figures:
tables:
status: outline
---

# Section purpose

# Claims to include

# Evidence
```

Draft text

Citations needed

Figures/tables needed

What this does for the work: makes Bundle 9 ready to feed Bundle 10 without turning the vault itself into the LaTeX build system.

prepare_figure_export_paths

Creates figure folders that mirror both the PKM and platform artifact structure.

Recommended links:

```
<vault>/50_figures/grn
  relates to /workspace/artifacts/nca-art-grn/figures

<vault>/50_figures/nca_art
  relates to /workspace/artifacts/nca-art-grn/figures

<vault>/50_figures/platform
  relates to /workspace/artifacts/platform/figures

<vault>/50_figures/agentfield
  relates to /workspace/artifacts/agentfield/figures

<vault>/50_figures/paperclip
  relates to /workspace/artifacts/paperclip/figures
```

It should create directories, not copy figures by default.

What this does for the work: gives figures a predictable bridge from experiments to paper-writing.

prepare_latex_template_binding

Creates metadata that says which vault notes map to which LaTeX project.

Recommended path:

```
<vault>/70_bridge/latex/grn-paper-binding.yaml
```

Example:

```
paper: grn-paper
latex_project: /workspace/artifacts/papers/grn-paper
```

```
publish_kasten: /workspace/pkm/zettelkasten/30_publish_kasten
section_notes: /workspace/pkm/zettelkasten/30_publish_kasten/sequences
figures: /workspace/pkm/zettelkasten/50_figures/grn
bibliography: /workspace/artifacts/papers/grn-paper/bib/grn.bib
```

What this does for the work: lets the Publisher bundle know where to look when mapping alloy notes into the LaTeX manuscript.

Acceptance criteria

The following should be valid managed setup commands:

```
sudo config --target publisher bootstrap step prepare_obsidian_vault_access
sudo config --target publisher bootstrap step prepare_atomic_zettelkasten_structure
sudo config --target publisher bootstrap step prepare_source_note_templates
sudo config --target publisher bootstrap step prepare_atom_note_templates
sudo config --target publisher bootstrap step prepare_molecule_note_templates
sudo config --target publisher bootstrap step prepare_topic_question_templates
sudo config --target publisher bootstrap step prepare_alloy_publish_note_templates
sudo config --target publisher bootstrap step prepare_latex_section_note_templates
sudo config --target publisher bootstrap step prepare_figure_export_paths
sudo config --target publisher bootstrap step prepare_latex_template_binding
sudo config --target publisher bootstrap step check_obsidian_vault_access
```

They must:

```
create the vault folder structure
create templates only if missing
create bridge metadata only if missing
not overwrite existing notes
not delete fleeting notes automatically
not print note contents
not call remote or local models
not index the vault
not build a paper
not modify the LaTeX project except through explicit binding metadata
```

Proposed tests

Syntax / registry tests

```
bash --noprofile --norc -n /home/vmuser/.local/bin/config.sh
bash --noprofile --norc -n /home/vmuser/.local/lib/config-sh/installers.sh
```

```
config bootstrap steps | grep prepare_atomic_zettelkasten_structure
config bootstrap steps | grep prepare_source_note_templates
config bootstrap steps | grep prepare_atom_note_templates
config bootstrap steps | grep prepare_molecule_note_templates
config bootstrap steps | grep prepare_alloy_publish_note_templates
config bootstrap steps | grep prepare_latex_section_note_templates
```

Dry structure tests

```
sudo config --target publisher bootstrap step prepare_atomic_zettelkasten_structure

test -d /workspace/pkm/zettelkasten/00_inbox
test -d /workspace/pkm/zettelkasten/10_reference_manager_kasten
test -d /workspace/pkm/zettelkasten/20_knowledge_kasten
test -d /workspace/pkm/zettelkasten/30_publish_kasten
test -d /workspace/pkm/zettelkasten/40_experiments
test -d /workspace/pkm/zettelkasten/50_figures
test -d /workspace/pkm/zettelkasten/60_templates
test -d /workspace/pkm/zettelkasten/70_bridge
```

Template tests

```
sudo config --target publisher bootstrap step prepare_source_note_templates
sudo config --target publisher bootstrap step prepare_atom_note_templates
sudo config --target publisher bootstrap step prepare_molecule_note_templates
sudo config --target publisher bootstrap step prepare_alloy_publish_note_templates

test -f /workspace/pkm/zettelkasten/60_templates/source_note.md
test -f /workspace/pkm/zettelkasten/60_templates/atom_note.md
test -f /workspace/pkm/zettelkasten/60_templates/molecule_note.md
test -f /workspace/pkm/zettelkasten/60_templates/alloy_note.md
```

LaTeX bridge tests

```
sudo config --target publisher bootstrap step prepare_latex_section_note_templates
sudo config --target publisher bootstrap step prepare_latex_template_binding

test -f /workspace/pkm/zettelkasten/60_templates/latex_section_note.md
test -f /workspace/pkm/zettelkasten/70_bridge/latex/grn-paper-binding.yaml
```

Non-overwrite test

```
echo "DO NOT OVERWRITE" > /workspace/pkm/zettelkasten/60_templates/atom_note.md
sudo config --target publisher bootstrap step prepare_atom_note_templates
grep "DO NOT OVERWRITE" /workspace/pkm/zettelkasten/60_templates/atom_note.md
```

Access check test

```
sudo config --target publisher bootstrap step check_obsidian_vault_access
```

Expected output should report structure status, but not print note bodies.

Summary

Bundle 9 should become the **knowledge operating system** for the research platform.

It prepares:

```
source capture  
atomic concepts  
insight molecules  
topic and research-question maps  
publishable alloy notes  
figure paths  
LaTeX section-note bridge
```

It supports your real work:

```
GRN discovery research  
NCA-ART theory and implementation  
Runpod experiment planning  
Agentfield architecture  
Paperclip adapter design  
OpenClaw / local model PKM reasoning  
MRes paper drafting
```

Its boundary is clear:

```
Bundle 9 organizes knowledge.  
Bundle 10 builds the LaTeX paper.  
OpenClaw may later query the vault.  
Agentfield may later reference research questions and experiment notes.  
Paperclip may later expose selected notes, summaries, runs, and review tasks.
```

So Bundle 9 is not “notes on the side.” It is the bridge from thinking to experiment design to publishable scientific argument.